Natural Language Processing

Fundamentals Post-Read Guide

For Second Year Computer Science Students

Academic Year 2024-25

Post-Lecture Overview

This guide introduces fundamental concepts of Natural Language Processing that form the foundation for understanding how computers process and understand human language. Read this before your NLP lectures to get familiar with the core concepts.

What You'll Revise:

Part 1 - NLP Fundamentals:

- ✓ What is Natural Language Processing
- ✓ Why computers need numbers instead of text
- ✓ The complete NLP pipeline
- ✓ Real-world applications

Part 2 - Text Preprocessing & Tokenization:

- ✓ Text cleaning techniques
- ✓ Breaking text into tokens
- ✓ Normalization methods
- ✓ Practical implementations

Part 3 - Vectorization & Embeddings:

- ✓ Converting text to numerical vectors
- ✓ Bag of Words and TF-IDF
- ✓ Word embeddings and semantic relationships
- ✓ Modern transformer architectures

Estimated reading time: 35 - 45 minutes

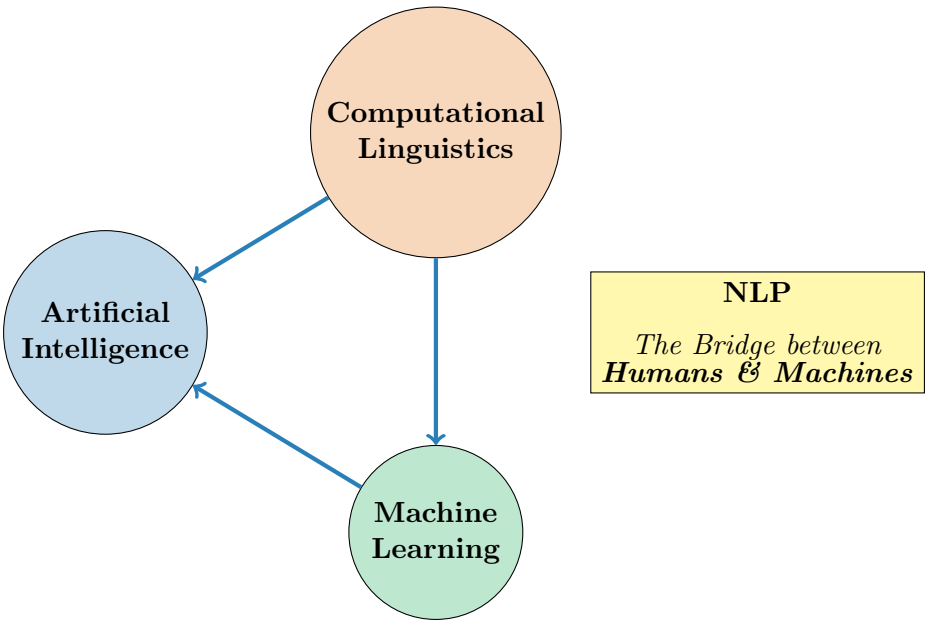
1 What is Natural Language Processing?

Definition

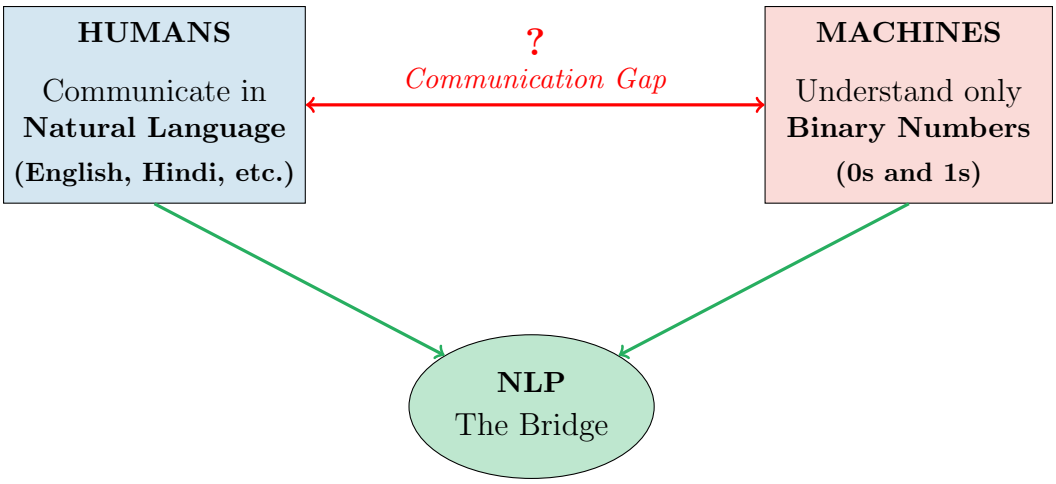
Natural Language Processing (NLP) is a branch of Artificial Intelligence that enables computers to understand, interpret, and generate human language in meaningful ways.

1.1 The Foundation of NLP

NLP sits at the intersection of three core disciplines:

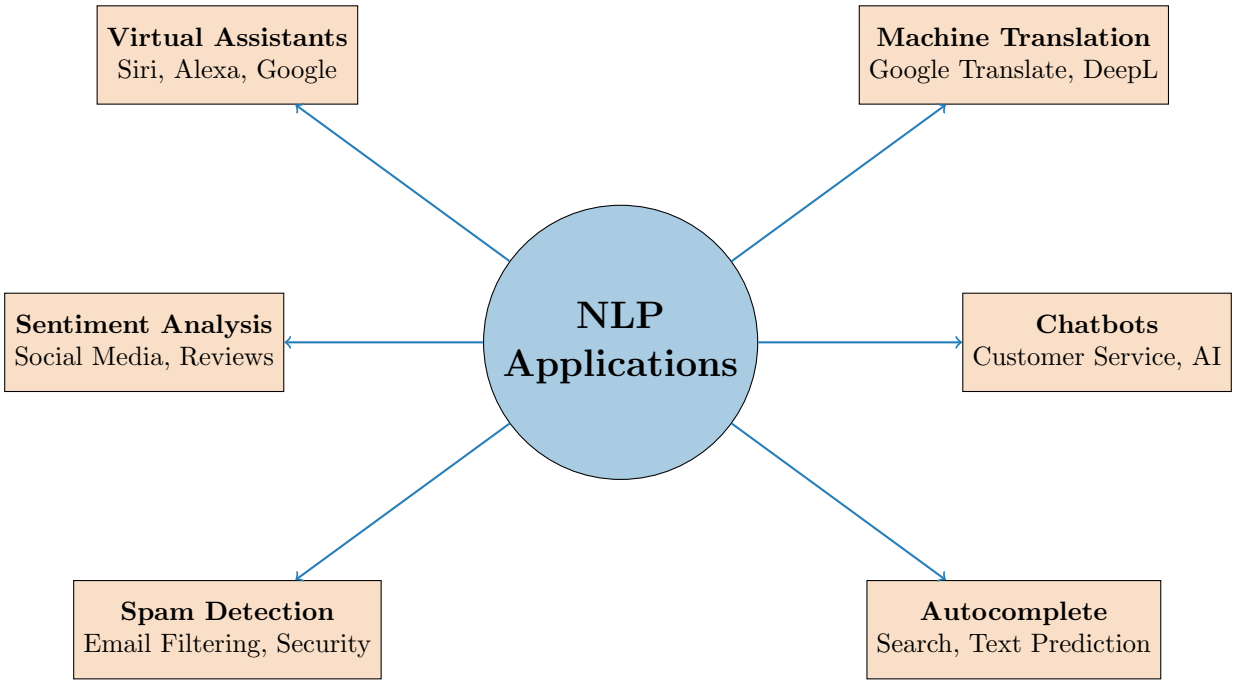


1.2 The Human-Machine Communication Gap



Key Challenge: Computers process numbers, humans communicate with words. NLP translates between these two worlds.

1.3 Real-World NLP Applications



1.4 How NLP Works: A Simple Example

Example: Google Search "best pizza near me"

What happens behind the scenes:

1. **Tokenization:** Break into tokens → [`"best"`, `"pizza"`, `"near"`, `"me"`]
2. **Intent Recognition:** Understand this is a food search query
3. **Entity Extraction:** Identify `"pizza"` = food, `"near me"` = location
4. **Location Processing:** Use GPS coordinates from device
5. **Ranking:** Find and rank relevant pizza restaurants
6. **Results:** Display top-rated options nearby

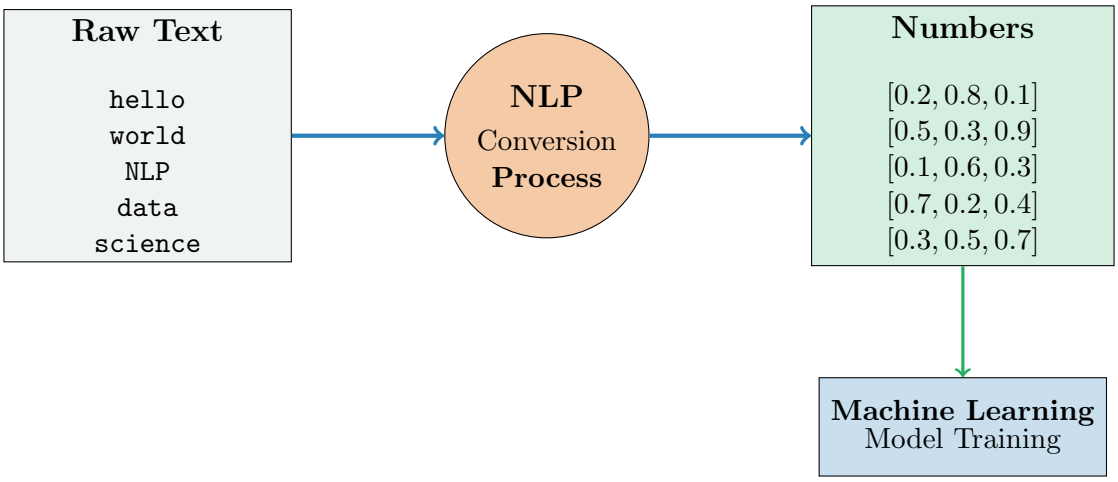
Without NLP: You would need to manually write complex database queries!

2 Why NLP Needs Numbers

2.1 The Fundamental Problem

Computers Only Understand NUMBERS

But we communicate with WORDS



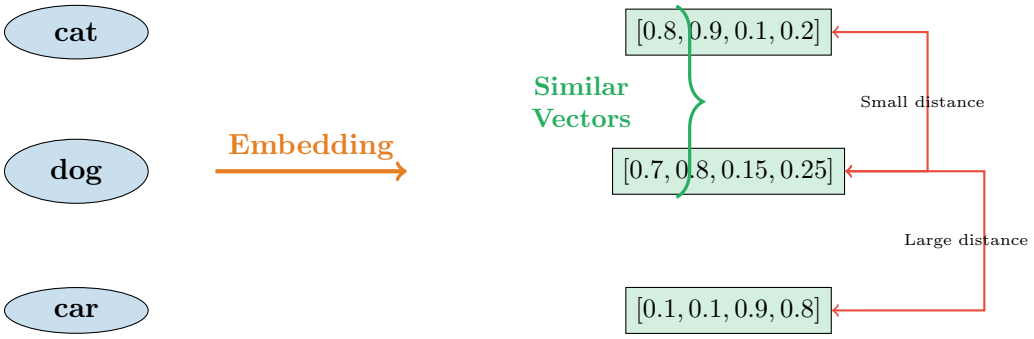
2.2 What Happens Without Numerical Conversion?

Problems Without Numbers

- **No Calculations:** Cannot perform mathematical operations on raw text
- **No Similarity Measurement:** Cannot measure how similar words are
- **No Machine Learning:** ML algorithms require numerical input
- **No Pattern Recognition:** Cannot identify patterns in text data
- **No Predictions:** Cannot train models to make forecasts

2.3 The Word Embedding Concept

Transform text into vectors that **preserve meaning and relationships**.



Notice: Words with similar meanings (*cat*, *dog*) have similar vectors, while different concepts (*car*) have different vectors.

2.4 How Vector Similarity is Calculated

To measure how similar two vectors are, we use **Cosine Similarity**:

Cosine Similarity Formula

$$\text{Similarity}(A, B) = \frac{A \cdot B}{||A|| \times ||B||} = \frac{\sum_{i=1}^{n} A_i \times B_i}{\sqrt{\sum_{i=1}^{n} A_i^2} \times \sqrt{\sum_{i=1}^{n} B_i^2}}$$

Result ranges from:

- **+1** = Identical vectors (perfectly similar)
- **0** = Orthogonal vectors (no similarity)
- **-1** = Opposite vectors (completely different)

Step-by-Step Calculation Example:

Let's calculate similarity between *cat* and *dog* vectors:

- **cat vector:** $A = [0.8, 0.9, 0.1, 0.2]$
- **dog vector:** $B = [0.7, 0.8, 0.15, 0.25]$

Step 1: Calculate Dot Product $(A \cdot B)$

$$\begin{aligned} A \cdot B &= (0.8 \times 0.7) + (0.9 \times 0.8) + (0.1 \times 0.15) + (0.2 \times 0.25) \\ &= 0.56 + 0.72 + 0.015 + 0.05 \\ &= 1.345 \end{aligned}$$

Step 2: Calculate Magnitude of A $(||A||)$

$$\begin{aligned} ||A|| &= \sqrt{0.8^2 + 0.9^2 + 0.1^2 + 0.2^2} \\ &= \sqrt{0.64 + 0.81 + 0.01 + 0.04} \\ &= \sqrt{1.50} = 1.225 \end{aligned}$$

Step 3: Calculate Magnitude of B ($||B||$)

$$\begin{aligned} ||B|| &= \sqrt{0.7^2 + 0.8^2 + 0.15^2 + 0.25^2} \\ &= \sqrt{0.49 + 0.64 + 0.0225 + 0.0625} \\ &= \sqrt{1.215} = 1.102 \end{aligned}$$

Step 4: Calculate Cosine Similarity

$$\begin{aligned} \text{Similarity}(cat, dog) &= \frac{1.345}{1.225 \times 1.102} \\ &= \frac{1.345}{1.350} \\ &= 0.996 \end{aligned}$$

Result: Similarity = **0.996** (very close to 1)
This means *cat* and *dog* vectors are **highly similar**, which makes sense because they are both animals!

Compare with cat and car:

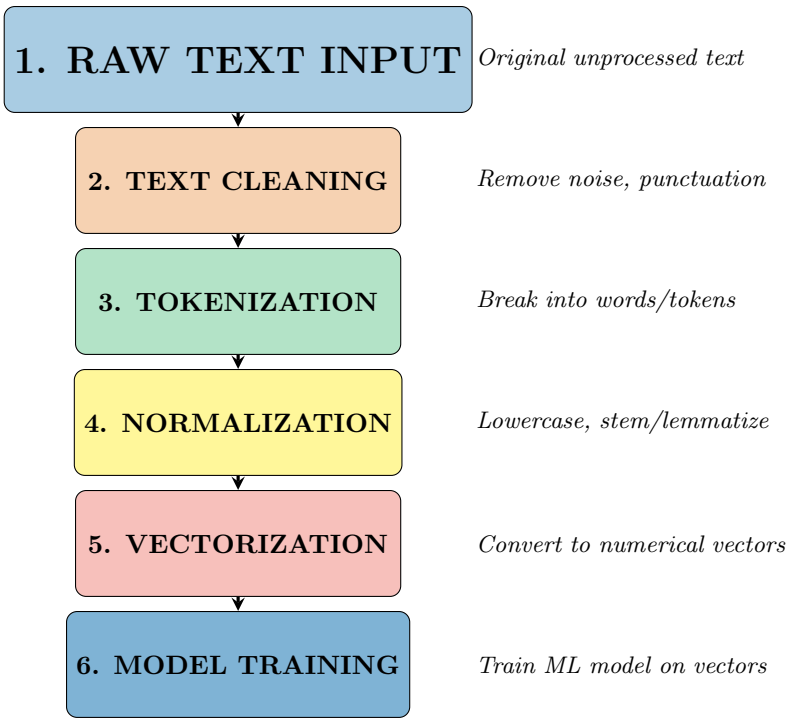
If we calculate similarity between *cat* $[0.8, 0.9, 0.1, 0.2]$ and *car* $[0.1, 0.1, 0.9, 0.8]$:

$$\begin{aligned} \text{Similarity}(cat, car) &= \frac{(0.8 \times 0.1) + (0.9 \times 0.1) + (0.1 \times 0.9) + (0.2 \times 0.8)}{||cat|| \times ||car||} \\ &= \frac{0.08 + 0.09 + 0.09 + 0.16}{1.225 \times 1.216} \\ &= \frac{0.42}{1.490} = 0.282 \end{aligned}$$

Result: Similarity = **0.282** (much lower)
This shows *cat* and *car* are **less similar**, as they represent different concepts!

3 The Complete NLP Pipeline

Every NLP task follows a systematic pipeline:



3.1 Pipeline Example: Processing Real Text

Example: Step-by-Step Transformation

Input Text: `"Hello World!  NLP is #1 @ preprocessing!!!"`

Stage	Output
1. RAW TEXT INPUT	`Hello World!  NLP is #1 @ preprocessing!!!`
2. TEXT CLEANING	`hello world nlp is 1 preprocessing`
3. TOKENIZATION	`['hello', 'world', 'nlp', 'is', '1', 'preprocessing']`
4. NORMALIZATION	`['hello', 'world', 'nlp', 'preprocessing']`
5. VECTORIZATION	`[[0.2, 0.8], [0.5, 0.3], [0.1, 0.9], [0.7, 0.4]]`
6. MODEL TRAINING	`Trained model ready for predictions`

Explanation:

- **Stage 2 (Cleaning):** Lowercase + remove punctuation + remove extra spaces
- **Stage 4 (Normalization):** Remove stopwords like "is" and "1"
- **Stage 5 (Vectorization):** Each word converted to numerical vector
- **Stage 6 (Model Training):** Vectors fed into ML algorithm

Result: Clean numerical representation ready for machine learning!

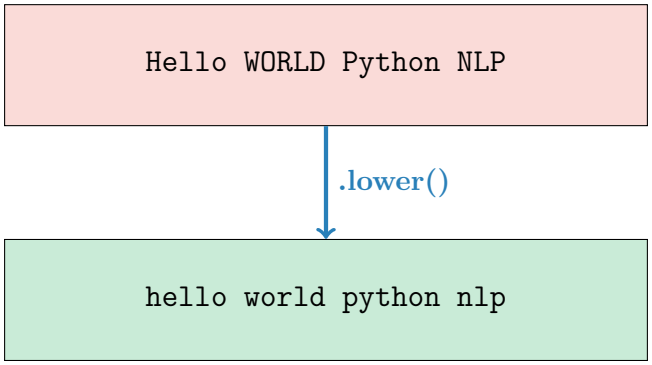
4 Text Preprocessing Techniques

Goal of Preprocessing

Clean and standardize text to make it ready for machine learning models. **Remove noise, keep signal.**

4.1 Technique 1: Lowercasing

Purpose: Treat "Hello", "HELLO", and "hello" as the same word.

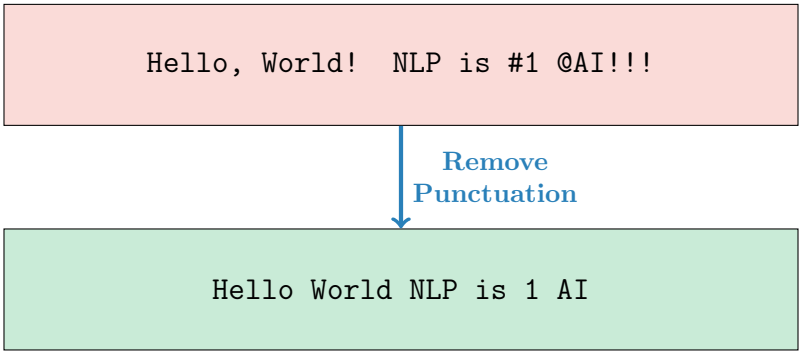



```python
text = "Hello WORLD! Python and NLP are AWESOME."
cleaned_text = text.lower()
print(cleaned_text)
# Output: hello world! python and nlp are awesome.
```

Listing 1: Lowercasing in Python

4.2 Technique 2: Removing Punctuation

Purpose: Remove special characters that don't add meaning.




```python
import string
text = "Hello, World! NLP is #1 @ AI!!!"
cleaned = text.translate(str.maketrans('', '', string.punctuation))
print(cleaned)
# Output: Hello World NLP is 1  AI
```

Listing 2: Removing Punctuation

4.3 Technique 3: Complete Cleaning Function

```python
import re
import string

def clean_text(text):
    # Convert to lowercase
    text = text.lower()

    # Remove URLs
    text = re.sub(r'http\S+|www\S+', '', text)

    # Remove punctuation
    text = text.translate(str.maketrans('', '', string.punctuation))

    # Remove extra whitespaces
    text = ' '.join(text.split())

    return text

# Example usage
raw = "Check OUT! Visit https://example.com. NLP is #Amazing!!!"
clean = clean_text(raw)
print(clean)
# Output: check out nlp is amazing
```

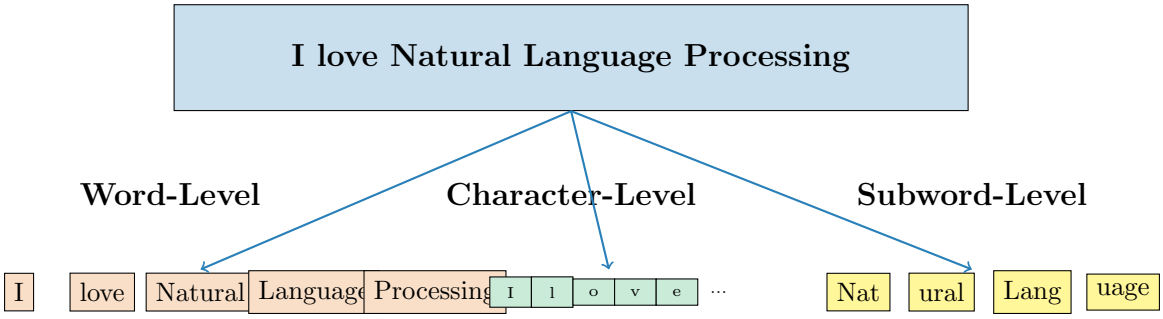
Listing 3: Complete Text Cleaning Pipeline

5 Tokenization: Breaking Text into Units

What is Tokenization?

Tokenization is the process of breaking text into smaller units called **tokens** (words, characters, or subwords).

5.1 Three Types of Tokenization



5.2 Comparison of Tokenization Methods

Method	Advantages	Disadvantages	Use Case
Word-Level	• Simple • Preserves meaning • Fast processing	• Large vocabulary • Can't handle rare words	Text classification, sentiment analysis
Character-Level	• Small vocabulary • Handles any word • Language agnostic	• Very long sequences • Loses word meaning	Typo correction, text generation
Subword-Level	• Balanced vocabulary • Handles rare words • Good for morphology	• Complex implementation • Needs training	Modern transformers, translation

5.3 Implementation Example

```python
from nltk.tokenize import word_tokenize

text = "Don't worry! NLP handles contractions, punctuation."
tokens = word_tokenize(text)
print(tokens)
# Output: ['Do', "n't", 'worry', '!', 'NLP', 'handles',
#          'contractions', ',', 'punctuation', '.']
```

Listing 4: Word Tokenization with NLTK

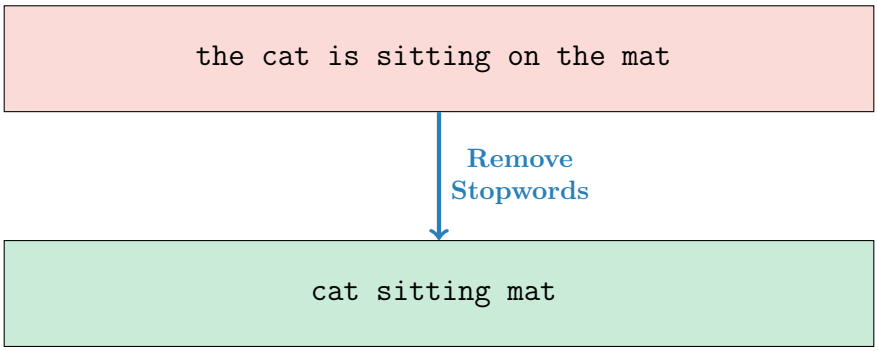
6 Text Normalization

What is Normalization?

Text Normalization converts text to a standard format by reducing words to their base forms. This reduces vocabulary size and groups similar words.

6.1 Stopwords Removal

Stopwords are common words with little meaning: "the", "is", "a", "an", "in", "on", etc.



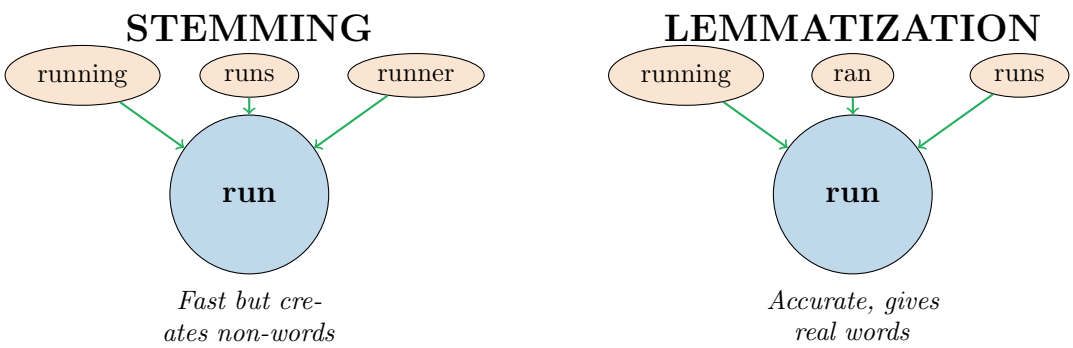
Removed: "the" (2×), "is", "on"

```python
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize

text = "The cat is sitting on the mat"
tokens = word_tokenize(text.lower())
stop_words = set(stopwords.words('english'))

filtered = [word for word in tokens if word not in stop_words]
print(filtered)
# Output: ['cat', 'sitting', 'mat']
```

Listing 5: Removing Stopwords

6.2 Stemming vs Lemmatization

Both reduce words to base forms, but use different approaches:



Choice Guide: Use **Stemming** for speed, **Lemmatization** for accuracy.

Word	Stemming	Lemmatization	Note
studies	studi	study	Lemma is proper word
better	better	good	Lemma finds true root
geese	gees	goose	Lemma handles irregulars

7 Converting Text to Numbers: Vectorization

7.1 Method 1: Bag of Words (BoW)

Count how many times each word appears in documents.

Doc 1: `I love NLP`

Doc 2: `NLP is amazing`

Doc 3: `I love ML`

$\xrightarrow{\textbf{BoW}}$

	I	love	NLP	is	amazing	ML
D1	1	1	1	0	0	0
D2	0	0	1	1	1	0
D3	1	1	0	0	0	1

Key Points: Each row = document, each column = word, values = counts.

```python
from sklearn.feature_extraction.text import CountVectorizer

docs = ["I love NLP", "NLP is amazing", "I love ML"]
vectorizer = CountVectorizer()
bow_matrix = vectorizer.fit_transform(docs)

print("Vocabulary:", vectorizer.get_feature_names_out())
print("BoW Matrix:\n", bow_matrix.toarray())
```

Listing 6: Bag of Words Implementation

7.2 Method 2: TF-IDF (Term Frequency-Inverse Document Frequency)

Weighs words by importance rather than just counting.

TF-IDF Formula

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \text{IDF}(t)$$

Where:

- **TF** = Term Frequency = $\frac{\text{Count of term in document}}{\text{Total terms in document}}$
- **IDF** = Inverse Document Frequency = $\log\left(\frac{\text{Total documents}}{\text{Documents containing term}}\right)$

7.3 Intuition Behind TF-IDF

High TF-IDF Score

Word appears **often**
in THIS document
but **rarely** in others

→ *Important for this document*

Example:
"neural network"
in ML research paper

Low TF-IDF Score

Word appears in
MANY documents
(common word)

→ *Less important*

Example:
"the", "is", "a"
(stopwords)

```python
from sklearn.feature_extraction.text import TfidfVectorizer

docs = ["The cat sits", "The dog plays", "The cat and dog"]
tfidf = TfidfVectorizer()
matrix = tfidf.fit_transform(docs)

print("TF-IDF Matrix:\n", matrix.toarray().round(2))
# Higher values = more important for that document
```

Listing 7: TF-IDF Implementation

8 Word Embeddings: The Modern Approach

8.1 Limitations of Traditional Methods

Problems with BoW & TF-IDF

- **Sparse Vectors:** Mostly zeros (inefficient storage)
- **No Semantic Meaning:** Can't capture word relationships
- **High Dimensionality:** Vector size = vocabulary size (10,000+)
- **No Similarity:** Can't tell "cat" and "dog" are related animals

8.2 What are Word Embeddings?

Definition

Word Embeddings are dense vector representations where words with similar meanings have similar vectors. Each word becomes a fixed-size vector (typically 100-300 dimensions) where position in space captures meaning.

8.3 Dense vs Sparse Comparison

Sparse (One-Hot)

Example: cat

`[0, 0, 1, 0, 0, ..., 0]`

- 10,000 dimensions
- 99.99% zeros
- No meaning captured
- Huge memory waste

Dense (Embedding)

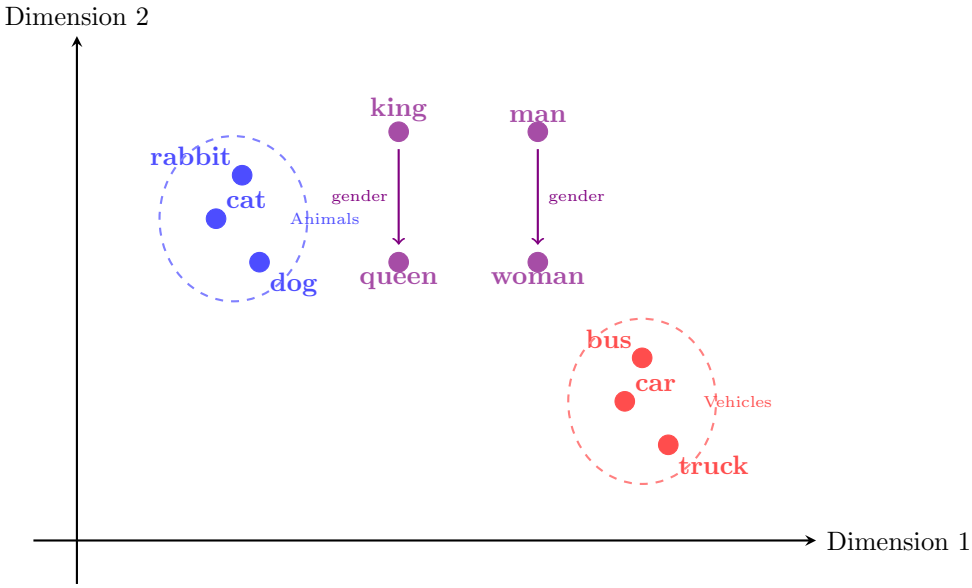
Example: cat

`[0.5, -0.3, 0.8, 0.1]`

- 100-300 dimensions
- All values meaningful
- Captures semantics
- Efficient storage

8.4 Semantic Relationships in Vector Space

Word embeddings capture meaning through vector geometry!



Famous Vector Arithmetic:
king - man + woman ≈ queen

8.5 Word2Vec: Learning Embeddings

Word2Vec learns by predicting context words:

```python
from gensim.models import Word2Vec

sentences = [
    ['i', 'love', 'natural', 'language', 'processing'],
    ['machine', 'learning', 'is', 'amazing'],
    ['deep', 'learning', 'neural', 'networks']
]

model = Word2Vec(sentences, vector_size=100, window=5,
                 min_count=1, epochs=100)

# Get vector for a word
vector = model.wv['learning']
print("Vector (first 5 dims):", vector[:5])

# Find similar words
similar = model.wv.most_similar('learning', topn=3)
print("Similar words:", similar)
```

Listing 8: Training Word2Vec Embeddings

9 Complete NLP Pipeline: Sentiment Analysis

Project Goal: Predict if movie reviews are positive or negative.

9.1 Step 1: Data Preparation

```python
import pandas as pd

data = {
    'review': [
        "This movie was excellent! Great acting.",
        "Terrible waste of time. Very boring.",
        "Amazing story. Loved it!",
        "Worst movie ever.",
        "Fantastic performances.",
        "Horrible script.",
        "Best movie of the year!",
        "Awful experience."
    ],
    'sentiment': [1, 0, 1, 0, 1, 0, 1, 0]  # 1=positive, 0=negative
}

df = pd.DataFrame(data)
```

Listing 9: Create Dataset

9.2 Step 2: Text Preprocessing

```python
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
import re

class TextPreprocessor:
    def __init__(self):
        self.stop_words = set(stopwords.words('english'))
        self.lemmatizer = WordNetLemmatizer()

    def preprocess(self, text):
        text = text.lower()  # Lowercase
        text = re.sub(r'[^a-z\s]', '', text)  # Remove special chars
        tokens = text.split()  # Tokenize
        tokens = [self.lemmatizer.lemmatize(w)
                  for w in tokens if w not in self.stop_words]
        return ' '.join(tokens)

preprocessor = TextPreprocessor()
df['cleaned'] = df['review'].apply(preprocessor.preprocess)
```

Listing 10: Preprocessing Class

9.3 Step 3: Vectorization & Model Training

```python
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    df['cleaned'], df['sentiment'], test_size=0.25)

```

```python
# Vectorize with TF-IDF
vectorizer = TfidfVectorizer(max_features=100)
X_train_vec = vectorizer.fit_transform(X_train)
X_test_vec = vectorizer.transform(X_test)

# Train model
classifier = MultinomialNB()
classifier.fit(X_train_vec, y_train)

# Evaluate
accuracy = classifier.score(X_test_vec, y_test)
print(f"Accuracy: {accuracy:.2%}")
```

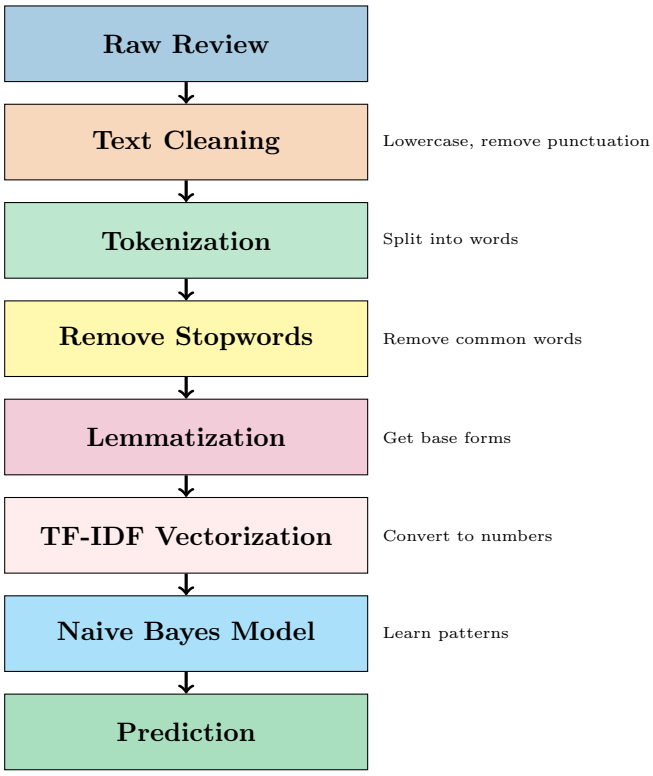
Listing 11: Train Classifier

9.4 Step 4: Make Predictions

```python
def predict_sentiment(text):
    cleaned = preprocessor.preprocess(text)
    vec = vectorizer.transform([cleaned])
    prediction = classifier.predict(vec)[0]
    probability = classifier.predict_proba(vec)[0]

    sentiment = "Positive" if prediction == 1 else "Negative"
    confidence = max(probability)
    return sentiment, confidence

# Test
new_reviews = [
    "This film was absolutely brilliant!",
    "Complete disaster, avoid at all costs"
]

for review in new_reviews:
    sentiment, conf = predict_sentiment(review)
    print(f"{review}\n-> {sentiment} ({conf:.1%})\n")
```

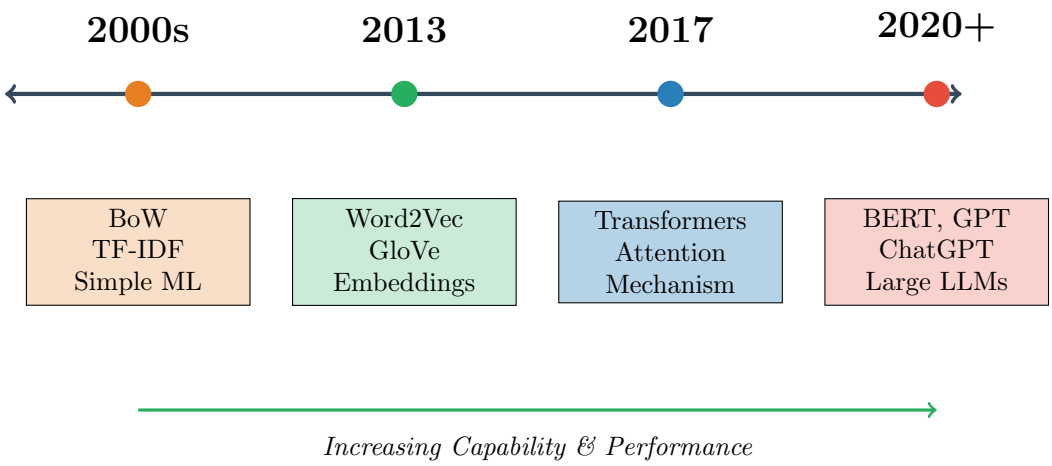
Listing 12: Predict on New Reviews

9.5 Complete Pipeline Visualization



10 Modern NLP: Transformers & Deep Learning

10.1 Evolution of NLP Technologies



10.2 What are Transformers?

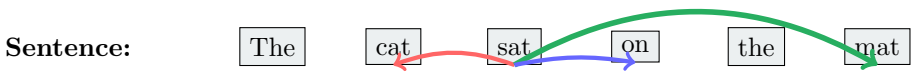
Transformers Explained

Transformers are neural network architectures that use *attention mechanisms* to process text. Unlike older methods that process words sequentially, transformers can understand relationships between **all words simultaneously**.

10.3 Key Innovation: Attention Mechanism

The Problem: Older models process words one-by-one and lose long-range dependencies.

The Solution: Attention looks at all words at once and learns which ones matter.



Attention weights show:

- Word "`sat`" pays **high attention** to "`mat`" (where?)
- Moderate attention to "`cat`" (who?) and "`on`" (relation)
- This captures semantic relationships!

10.4 Transformer Architecture Overview

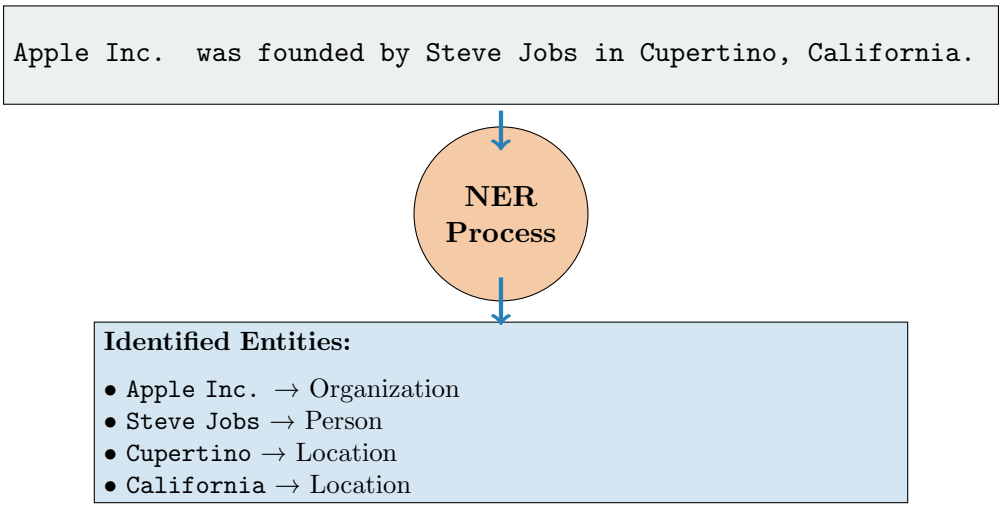
10.5 Popular Transformer Models

Model	**Year**	**Key Feature**
BERT	2018	Bidirectional understanding, excellent for classification
GPT-3/4	2020/23	Autoregressive, exceptional text generation
T5	2019	Text-to-text framework, versatile for many tasks
ChatGPT	2022	Conversational AI, instruction following

11 Advanced NLP Concepts

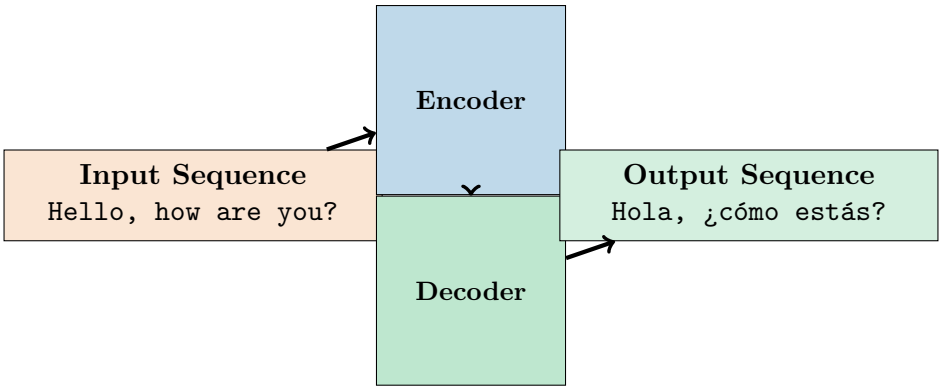
11.1 Named Entity Recognition (NER)

Identify and classify named entities in text.



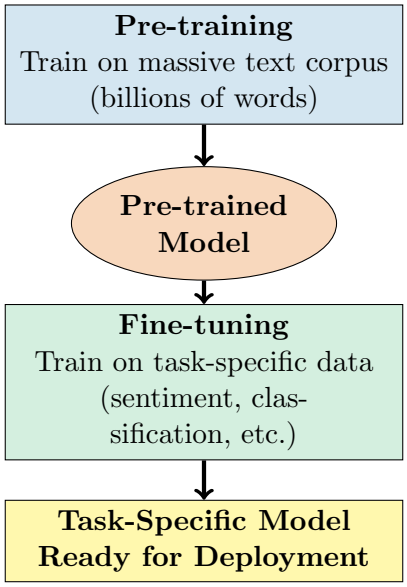
11.2 Sequence-to-Sequence Models

Transform one sequence into another (e.g., translation, summarization).



11.3 Transfer Learning in NLP

Use pre-trained models and fine-tune for specific tasks.


Pre-training
Train on massive text corpus (billions of words)
Pre-trained Model
Fine-tuning
Train on task-specific data (sentiment, classification, etc.)
Task-Specific Model Ready for Deployment

12 Real-World NLP Applications

12.1 Application 1: Text Classification

Categorize text into predefined classes.

Examples:

- Spam detection in emails
- Sentiment analysis of product reviews
- Topic categorization of news articles
- Language identification

12.2 Application 2: Information Extraction

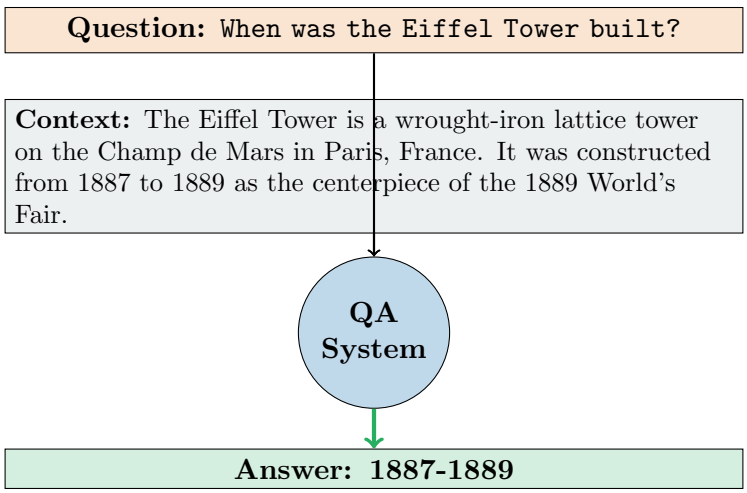
Extract structured information from unstructured text.

Examples:

- Extract dates, locations, and people from documents
- Identify relationships between entities
- Build knowledge graphs from text
- Resume parsing and information extraction

12.3 Application 3: Question Answering

Build systems that answer questions based on text.



12.4 Application 4: Machine Translation

Translate text from one language to another.

Key Challenges:

- Handling idioms and cultural expressions
- Maintaining context across long sentences
- Dealing with grammatical differences
- Preserving meaning and tone

12.5 Application 5: Text Summarization

Generate concise summaries of longer texts.

Two Approaches:

1. **Extractive Summarization:** Select important sentences from original text
2. **Abstractive Summarization:** Generate new sentences that capture key points

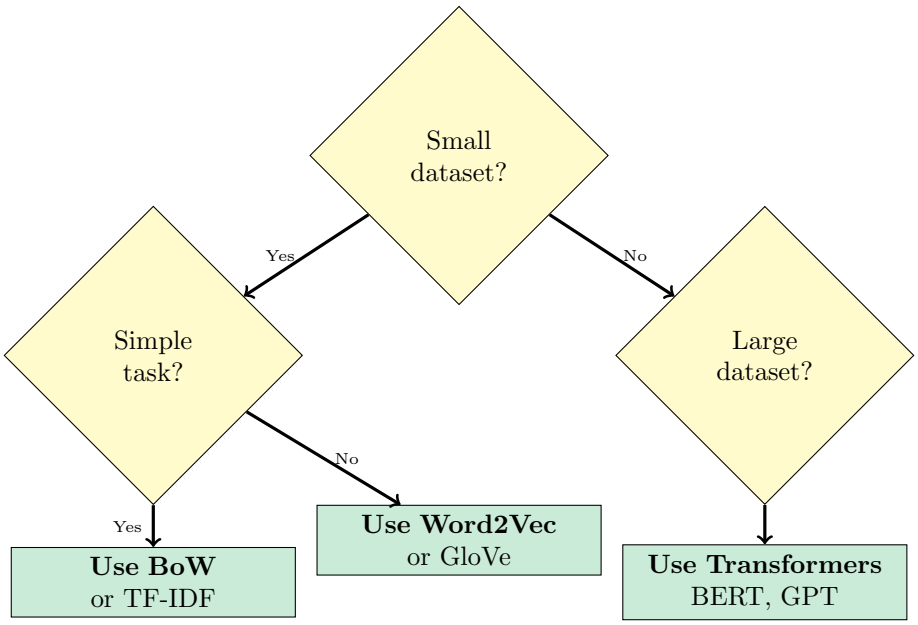
13 Summary & Best Practices

13.1 Key Concepts Recap

What You've Learned

1. **NLP Fundamentals:** Bridge between human language and machine understanding
2. **Why Numbers:** Computers need numerical representations to process text
3. **Text Preprocessing:** Clean, tokenize, and normalize text systematically
4. **Vectorization Methods:** BoW, TF-IDF, and Word Embeddings
5. **Complete Pipeline:** From raw text to trained ML models
6. **Modern Approaches:** Transformers and attention mechanisms
7. **Real Applications:** Classification, extraction, translation, and more

13.2 Decision Guide: Which Method to Use?



13.3 Best Practices Checklist

Essential Best Practices

- ☐ Explore data first
- ☐ Start simple
- ☐ Clean consistently
- ☐ Split data properly
- ☐ Same preprocessing everywhere
- ☐ Handle unknown words
- ☐ Validate separately
- ☐ Monitor overfitting
- ☐ Document pipeline
- ☐ Consider resources

13.4 Common Pitfalls & Solutions

Mistake	Problem	Solution
Fitting vectorizer on test data	Data leakage	Fit only on training data
Ignoring rare words	Model fails on new vocabulary	Use subword tokenization
Over-preprocessing	Loss of information	Balance cleaning with preservation
Ignoring class imbalance	Biased predictions	Use balanced datasets or weighting

13.5 Next Steps for Learning

1. **Practice Implementation:** Build projects from scratch
2. **Join Competitions:** Kaggle NLP challenges
3. **Read Research Papers:** Stay updated with latest advances
4. **Experiment:** Try different preprocessing techniques
5. **Build Real Projects:**
   - Sentiment analysis system
   - Text classification API
   - Chatbot with NLP
   - Named entity recognition tool

13.6 Essential Resources

Learning Resources

- **Python Libraries:** NLTK, spaCy, Gensim, Hugging Face Transformers
- **Online Courses:** Stanford CS224N, fast.ai NLP course, DeepLearning.AI
- **Books:**
  - "Speech and Language Processing" by Jurafsky & Martin
  - "Natural Language Processing with Python" by Bird et al.
  - "Transformers for Natural Language Processing" by Rothman
- **Practice Platforms:** Kaggle, GitHub, Google Colab, Hugging Face

13.7 Final Thoughts

Remember

NLP is about **understanding** and **representing** human language in ways machines can process effectively.

Start with fundamentals, practice consistently, and gradually explore advanced techniques. The field is rapidly evolving — stay curious and keep learning!

Practice is the key to mastery!

Good luck with your NLP journey!

End of Pre-Read Material

You are now ready for your NLP lectures. Review this material before class, and refer back to it as needed during your coursework. Keep practicing, and don't hesitate to experiment with the code examples!